Learning-Augmented KV-Cache Retention for Tool-Using LLM Agents

Group 9 — David Mei · Omkar Lashkare · Brian Park · Zijie Shen · Hangting Lu

Objective and Project Description

Our project investigates when an LLM serving system should retain a tool-using agent's key–value (KV) cache during a tool-call pause. Retaining avoids reprocessing a long context when the agent returns but occupies scarce GPU memory; evicting frees capacity yet incurs recomputation and queueing delay. We formulate the choice as online ski rental: the pause duration is unknown, holding costs accumulate until the agent returns or the cache is evicted, and eviction incurs a one-time penalty if the pause continues. Our learning-augmented policy uses a predicted return time and a trust parameter λ to choose retention within a bounded range around the prediction-free break-even time $b = B/c$. Unlike an ordinary prediction-based TTL, the contribution is the explicit consistency–robustness trade-off together with evaluation of **when prediction actually helps** — we treat prediction quality as an independent variable, which no prior system has done. Theory addresses a single pause; a trace-driven, capacity-constrained simulator addresses concurrent load. Continuum-style TTL is compared under identical traces, budgets, and cost assumptions.

Timeline and Expected Deliverables

The plan spans ten weeks with buffers in Weeks 2, 5, and 8. Weeks 1–2 cover literature, research question, novelty gap, and evaluation design; Week 3 processes real agent traces; Week 4 builds duration predictors; Week 5 delivered the progress presentation; Weeks 6–7 develop the retention rule, simulator, baselines, and initial sweeps; Week 8 stabilizes these and studies prediction failure. The repository documents work through Week 8. Week 9 priorities are hardware cost calibration, predictor precision work, and validating the scarce-memory experiments. Week 10 is reserved for final figures, reproducible code, the report, and the presentation. Percentages below estimate readiness of each promised output, not hours spent.

Deliverable Progress and Completion Plans

Literature review and processed traces — 100% complete. The novelty review covers Continuum, TraceLab, InferCept, SAGA, ConServe, Choi & Joshi, and learning-augmented ski-rental. We built and audited the TraceLab pause pipeline: the canonical table holds 357,161 records across 33 columns, of which 351,497 meet the upstream completeness criteria; Version 2 corrected roughly 32,000 mismeasured Codex pauses. Scripts, provenance checks, plots, and documentation are committed. Remaining work is carrying these definitions into the final report.

Workload characterization and tool-duration prediction — 95% complete. We evaluated 311,764 supervised pauses with session-level splits to prevent leakage across turns. The selected gradient-boosted median predictor reaches a test log1p-MAE of 0.5441, a significant gain over the per-tool median. The analysis then exposed a **mismatch between duration accuracy and the retention decision**: the policy consumes only one bit — whether the pause outlasts b — and on that bit the model had 22.3% recall, because log-error is dominated by the 95% of pauses that are short. Retraining directly on $P(T \geq b)$, with the acting threshold *derived* from the cost asymmetry (29:1, giving $p^* = 0.033$) rather than set to 0.5, raised recall to 87.0% but dropped precision to 14.6%. The fix was **session history**, which the Week 4 feature set omitted entirely: a long pause makes the next pause in that session 6.15× more likely to be long (37.5% against a 6.1% base rate; lag-1 autocorrelation 0.396). Adding it lifted average precision from 0.48 to 0.62 and roughly doubled precision at the operating point. The signal was present, leakage-free, and simply never supplied.

Learning-augmented policy and theory — 90% complete. The single-pause cost model, break-even fallback, and λ-policy are implemented, confining the timeout to $[\lambda b, b/\lambda]$; at $\lambda = 0.5$ the worst-case bound is 3× the clairvoyant single-pause optimum, and probes confirm the implementation attains $1 + 1/\lambda$ exactly at both tight cases

without exceeding it. Both fixed and cache-size-proportional occupancy costs are supported, and **measuring both settled an open question from Week 1**: under fixed c the break-even time spans 622× across requests (3.5 s to 209 s), while under proportional c it is identically 75.0 s and a fixed TTL becomes exactly equivalent to the break-even rule. Remaining: a concise proof, stated as applying to the single-pause model and not to completion time under load.

Trace-driven simulator and baselines — 90% complete. The discrete-event simulator models a bounded KV pool, LRU victim selection under pressure, and serialized recomputation. Seven retention policies are compared as interchangeable functions over identical traces: immediate eviction, always-retain, fixed TTL, break-even TTL, reconstructed Continuum-style TTL, our λ-policy, and a per-pause clairvoyant oracle. (LRU is the simulator's victim-selection rule, applying once memory is full, not a retention rule.) Instrumentation revealed that at a 150k-token budget 47.6% of admissions exceed total capacity, so those runs measure an undersized pool rather than a policy and are excluded. Remaining is the oracle's interpretation: minimizing single-pause cost does not minimize system latency, so we will report a per-session bound or invoke the proposal's fallback.

Empirical evaluation — 80% complete. The repository contains a 19-condition prediction-error sweep across two cost models and sweeps over memory budget and concurrency. **The largest and most robust result is that immediate eviction — what vLLM and SGLang do today — increases job completion time by 6.4× to 14.7× over any retention policy, and is insensitive to the memory budget because it never holds anything.** Differences among retention policies are 1–4% by comparison. Within that band, the Week 4 median predictor made modeled cost 26.2% worse than ignoring it, against a 23.8% ceiling under perfect prediction. **With session-history features that reverses**: cross-validated over all 3,667 sessions, the prediction-aware policy beats prediction-free break-even by a statistically significant margin under both cost models (fixed +0.059, 95% CI [+0.016, +0.103]; proportional +0.050, [+0.019, +0.082]), and the result survives excluding the rows used to pick the operating threshold. The single inherited 384-session test split could not resolve this — its long-pause rate is 4.5% against 6.2% pooled, so it is unrepresentative on precisely the quantity the decision turns on. Under replicated load, always-retain with LRU still beats every voluntary-eviction policy and even the per-pause oracle (72,538 s vs 74,165 s), because eviction causes re-prefill and re-prefill, not memory, is the binding constraint. Remaining: hardware calibration and matched Continuum comparisons.

Reproducible artifacts and final communication — 85% complete. GitHub holds dataset scripts, prediction and simulation code, 40 passing tests, experiment configurations, tables, figures, presentation material, and a drafted report; every experiment records its configuration and source commit. Week 10 work is rerunning final analyses, pinning library versions, reconciling cross-platform numerical drift, and submitting report, slides, and code. Optional vLLM/GPU deployment is not required for the core deliverable.

Difficulties and Risks to On-Time Completion

The principal risk is establishing how far the simulation results generalize, not completing the implementation. TraceLab durations are heavy-tailed and the released traces are sanitized, so tool arguments are incomplete and trace-derived pauses are proxies for GPU cache residency rather than measurements. Rare long pauses dominate the decision even when average error is good, and some spread is irreducible: within identical (provider, tool, tool-count, input-size-decile) groups the p90/p10 duration ratio is still 67×, so session history helps as a proxy for hidden state rather than by removing it. Evaluation power is the other constraint — the cost metric is driven by the ~5% of pauses that are long, which is why conclusions about it need cross-validation rather than a single split. The sharpest sensitivity is the occupancy-cost definition: whether c scales with cache size changes the sign of the headline, and prefill speed and memory opportunity cost are not yet calibrated on hardware, so both "prediction helps" and "prediction does not help" remain supportable. Our Continuum baseline is a reconstruction, not the authors' implementation, so its numbers are not comparable to their published figures. Two further scope limits: the simulator assumes session affinity, and high-concurrency runs require synthetic replication of the session population. Week 9 prioritizes calibration and sensitivity and records these limits; Week 10 favors stable reproducible outputs over optional CPU-offload or vLLM integration.